Distributed LLM Inference & Serving Platform — Project Dossier

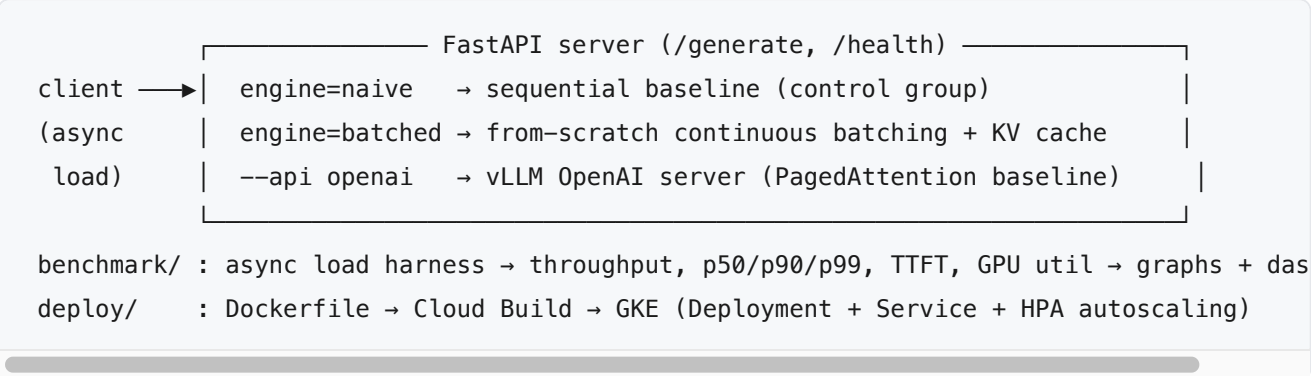
One line: A self-hosted LLM inference engine built three ways — a naive baseline, a from-scratch engine with a hand-written KV cache and continuous-batching scheduler, and a vLLM production baseline — benchmarked head-to-head on a GPU, with a quantization study and a Kubernetes autoscaling deployment.

Repo: https://github.com/JATAN2703/llm-inference-engine · **Model:** Qwen2.5-0.5B-Instruct **Stack:** Python, PyTorch, Hugging Face Transformers, vLLM, FastAPI, Docker, Google Cloud (Compute Engine GPU VMs, GKE, Cloud Build, Artifact Registry), Kubernetes (Deployment / Service / HPA), async load testing (httpx), matplotlib.

Problem

Naive LLM serving processes one request at a time, leaving the GPU mostly idle and making tail latency explode under load. Production engines fix this with KV caching, in-flight batching, paged memory, and quantization. This project **builds those internals from first principles** and **measures** them against a production baseline — so every claim is a number, not a slogan.

Architecture



What was built (8 phases)

0. **Scaffold** — clean repo, cached auto-device model loader (CUDA/MPS/CPU), smoke test.
1. **Naive baseline server** — FastAPI `/generate` with rigorous timing (TTFT, tokens/sec), deliberately serialized (a `threading.Lock`) as the honest control group.
2. **KV cache from scratch** — hand-written scaled-dot-product attention + a `KVCache` that appends K/V explicitly, plus a manual decode loop on the real model threading `past_key_values` with correct `cache_position`. Proves the $O(n^2) \rightarrow O(n)$ win and the memory formula.

3. **Continuous batching scheduler** — a single scheduler thread that admits/evicts sequences at decode -step boundaries, runs one batched forward across all in-flight requests, and manages a batched KV cache (left-padding + per-row positions/masks). Byte-identical to single-sequence greedy output.
4. **Benchmark harness** — async closed-loop load tester (httpx), concurrency sweep, percentile latency, GPU sampling (nvidia-smi), structured JSON/CSV, matplotlib graphs. Testable in-process via ASGI.
5. **vLLM baseline + quantization** — vLLM behind the same harness (OpenAI API); FP16/INT8/INT4 study measuring throughput, GPU memory, and quality (perplexity + FP16 token-agreement).
6. **Containerize + deploy** — Docker → Cloud Build → Artifact Registry → GKE with a Horizontal Pod Autoscaler; captured live autoscaling under load; scripted teardown for cost control.
7. **Dashboard + README** — self-contained HTML results dashboard and a benchmark-driven README.

Headline results

Throughput on one NVIDIA L4 (tokens/sec, 128-token generations)

engine	c=1	c=4	c=16	c=64
naive	32	35	34	35
batched (mine)	40	119	214	213
vLLM (PagedAttention)	48	177	655	2255

- **My batched engine ≈ 6× naive** (214 vs 34 tok/s at c=16); p99 latency stays bounded (~19s) while **naive's p99 explodes to 136s** at c=64 (requests serialize).
- **vLLM ≈ 3× my engine at c=16, ≈ 10× at c=64.** vLLM's paged KV cache packs far more sequences into VRAM (it reserves ~21.5 GB vs my ~1.3 GB) and keeps scaling where my contiguous left-padded cache saturates. My engine is the continuous-batching idea from scratch; vLLM is its paged, fragmentation -free evolution — and I can explain the gap precisely.

KV cache (from-scratch verification)

Toy attention: cached vs recompute work ratio grows **8.5×** → **32.5×** → **64.5×** with sequence length ($O(n^2) \to O(n)$). Real model: cached decode **~5× faster** with **byte-identical** tokens. Memory formula: `KV bytes = 2 · L · B · H_kv · d_head · S · bytes` → 24 MiB per 2048-token sequence (GQA: 2 KV heads).

Quantization study (bitsandbytes)

precision	tok/s	peak mem	perplexity	agreement vs FP16
FP16	27.8	980 MiB	14.0	1.00
INT8	6.9	640 MiB	14.2	0.64
INT4	17.7	489 MiB	25.6	0.07

Finding: on a 0.5B model, quantization **saved memory but not time** (dequant overhead dominates), and INT4 quality collapsed — the throughput payoff needs large models or native INT8/FP8 kernels.

Kubernetes autoscaling (GKE)

Under sustained load the HPA scaled pods **1** → **4** (CPU > 60% of request) and the cluster autoscaler **added a node**; capped cleanly at `maxReplicas`. Full teardown verified zero billing.

Key engineering decisions (what interviewers probe)

- **Engine separated from server** — internals are unit-testable without HTTP; engines are swappable behind one API and request schema.
- **Single scheduler thread owns the model** — HTTP handlers only enqueue and wait, so batched forward passes never race.
- **Contiguous left-padded batched cache** — correct and simple, and it makes padding/fragmentation *visible* — exactly the problem vLLM's paged cache solves. I built the version it improves on.
- **Measurement rigor & honesty** — closed-loop concurrency, client-side latency (captures queueing), p50/p90/p99; and I documented the caveats (vLLM workload not byte-identical; vLLM TTFT approximated; quant measured on T4). Naming a measurement's limits is the point, not hiding them.

What I learned

KV-cache memory arithmetic and why it (not compute) caps batch size; continuous batching and GPU utilization; PagedAttention and where it wins; the real quantization tradeoff; and the full container → GPU VM → GKE-autoscaling deployment path with strict cost control.

Interview narrative (say this)

> "I built an LLM inference engine three ways — a naive baseline, a from-scratch version with my own KV cache and continuous-batching scheduler, and a vLLM baseline — then benchmarked all three under load on an NVIDIA L4. My engine got ~6× the naive baseline's throughput;

> vLLM's PagedAttention pulled ~10× ahead of mine at high concurrency, and I can explain exactly why — paged, non-contiguous KV memory packs more sequences into VRAM than my contiguous cache. I also studied the FP16/INT8/INT4 quantization tradeoff and deployed the server on GKE with live autoscaling."